Modular Multi-Tenant CRM Platform

Data Model Strategy

Purpose

This document defines the data model strategy for the modular multi-tenant CRM platform.

The goal is to create a data architecture that supports:

- multi-tenant isolation
- shared platform infrastructure
- installable modules
- framework-agnostic CRM frontends
- flexible tenant customization
- long-term reporting and maintainability

This document focuses on how platform data, tenant data, core CRM data, and module-owned data should be structured.

---

1. Data Model Goals

The data model must support five major goals:

1. **Tenant isolation**
2. **Modular extensibility**
3. **Strong relational structure**
4. **Flexible customization**
5. **Operational simplicity**

The system should avoid two extremes:

- a rigid schema that makes module development painful
- an overly schemaless design that makes reporting, permissions, and data integrity difficult

The recommended direction is:

Recommended Strategy

Use a **relational core schema** with:

- **platform-level tables**
- **tenant-owned tables**
- **module-owned tables**
- **custom field support**
- **controlled JSON usage for metadata and configuration**

---

2. Data Ownership Layers

The platform data model should be divided into four logical layers.

2.1 Platform Layer

These tables support the platform itself and are not business records inside a tenant CRM.

Examples:

- users
- tenants
- memberships
- subscriptions
- plans
- modules
- module_versions
- tenant_modules
- domains
- audit_logs
- developer_accounts
- api_clients

This data is mostly platform-global, though some records relate to specific tenants.

2.2 Core CRM Layer

These are shared CRM primitives that many verticals will need.

Examples:

- contacts
- companies
- leads
- opportunities
- pipelines
- pipeline_stages
- tasks
- activities
- notes
- files
- tags

These are tenant-owned records and should always include `tenant_id`.

2.3 Module-Owned Layer

Modules may introduce structured business entities specific to an industry.

Examples:

- legal_cases
- legal_documents
- re_properties
- re_showings
- event_projects
- event_vendors
- healthcare_intakes

These records are still tenant-owned, but the tables are owned by modules.

2.4 Configuration and Metadata Layer

This layer stores configuration, schemas, mappings, layouts, and custom fields.

Examples:

- tenant_settings
- module_settings
- branding_settings
- custom_field_definitions
- custom_field_values
- saved_views
- automation_configs
- workflow_definitions

This layer powers flexibility without requiring one-off schema changes for every tenant preference.

---

3. Core Data Modeling Principles

3.1 Tenant-Owned Records Must Be Explicit

Every tenant-owned table should include:

- `id`
- `tenant_id`
- `created_at`
- `updated_at`

Often also:

- `created_by`
- `updated_by`
- `deleted_at` for soft deletes
- `status`
- `metadata` for non-critical structured attributes

Tenant ownership should never be implied only through parent joins if it can be made explicit.

3.2 Core Entities Should Be Reusable Across Modules

The platform should define common CRM primitives once and allow modules to relate to them.

For example:

- a Legal module should be able to link `legal_cases` to `contacts`
- a Real Estate module should be able to link `re_properties` to `contacts`
- an Event module should be able to link `event_projects` to `companies`

This keeps the system cohesive and improves cross-module reporting.

3.3 Module Tables Should Be Namespaced

Module-owned tables should follow a clear naming convention.

Recommended pattern:

- `legal_cases`
- `legal_documents`
- `re_properties`
- `re_showings`
- `event_projects`
- `event_vendors`

This prevents naming collisions and makes ownership clearer.

3.4 Use JSON Only for the Right Things

JSON is useful for:

- settings blobs
- UI layout preferences
- module metadata
- webhook payload snapshots
- rarely queried dynamic configuration

JSON should not be the primary storage strategy for important transactional business data that needs:

- filtering
- reporting
- joins

- indexing
- permissions
- validation

If the system needs to query or report on something regularly, it likely deserves relational structure.

---

4. Multi-Tenant Data Strategy

Recommended Strategy

Use a **shared database with tenant-scoped relational tables**.

Each tenant-owned record includes `tenant_id`.

4.1 Tenant Scope Rules

Every query against tenant-owned data must include tenant context.

Examples:

- `contacts.tenant_id`
- `tasks.tenant_id`
- `legal_cases.tenant_id`
- `re_properties.tenant_id`

This allows:

- simpler infrastructure
- centralized analytics
- consistent migrations
- easier module lifecycle management

4.2 Tenant Boundaries

The platform must clearly separate:

Platform-Global Records

Example:

- user identities
- module catalog
- subscription plans

Tenant-Specific Records

Example:

- contacts
- deals
- tasks
- module business records

Tenant-Scoped Configuration

Example:

- branding
- custom domains
- installed modules
- role assignments
- custom field definitions

This distinction should be reflected clearly in naming and schema design.

---

5. Core Platform Entities

Below are the foundational platform entities.

5.1 users

Represents an authenticated person in the platform.

Suggested Fields

- `id`
- `name`

- `email`
- `password_hash` or external auth identifier
- `avatar_url`
- `status`
- `last_login_at`
- `created_at`
- `updated_at`

A user is a platform identity, not a tenant-specific record.

5.2 tenants

Represents a CRM instance / organization environment.

Suggested Fields

- `id`
- `name`
- `slug`
- `status`
- `primary_domain`
- `timezone`
- `locale`
- `branding_id`
- `created_at`
- `updated_at`

A tenant is the top-level business container.

5.3 memberships

Connects users to tenants.

Suggested Fields

- `id`
- `tenant_id`
- `user_id`

- `role_id` or membership role reference
- `status`
- `invited_by`
- `joined_at`
- `created_at`
- `updated_at`

This allows users to belong to multiple tenants.

5.4 roles

Defines assignable roles.

Suggested Fields

- `id`
- `tenant_id` nullable if platform-default
- `name`
- `slug`
- `description`
- `is_system`
- `created_at`
- `updated_at`

Roles may be:

- platform-provided defaults
- tenant-customized roles

5.5 permissions

Defines individual permissions.

Suggested Fields

- `id`
- `key`
- `name`

- `scope`
- `module_slug` nullable
- `created_at`
- `updated_at`

Examples:

- `contacts.read`
- `contacts.write`
- `legal_cases.manage`
- `re_properties.read`

5.6 role_permissions

Maps permissions to roles.

Suggested Fields

- `id`
- `role_id`
- `permission_id`

5.7 domains

Stores tenant domains and subdomains.

Suggested Fields

- `id`
- `tenant_id`
- `host`
- `type`
- `status`
- `is_primary`
- `verified_at`
- `created_at`
- `updated_at`

5.8 modules

The module catalog.

Suggested Fields

- `id`
- `name`
- `slug`
- `description`
- `publisher_id`
- `category`
- `status`
- `is_public`
- `created_at`
- `updated_at`

5.9 module_versions

Stores versioned releases for modules.

Suggested Fields

- `id`
- `module_id`
- `version`
- `manifest_json`
- `compatibility_range`
- `release_notes`
- `published_at`
- `created_at`
- `updated_at`

5.10 tenant_modules

Represents modules installed into tenants.

Suggested Fields

- `id`
- `tenant_id`
- `module_id`
- `module_version_id`
- `status`
- `installed_by`
- `installed_at`
- `settings_json`
- `created_at`
- `updated_at`

5.11 subscriptions

Stores tenant subscription information.

Suggested Fields

- `id`
- `tenant_id`
- `plan_id`
- `status`
- `billing_provider`
- `external_subscription_id`
- `starts_at`
- `ends_at`
- `created_at`
- `updated_at`

5.12 audit_logs

Tracks important actions.

Suggested Fields

- `id`

- `tenant_id` nullable
- `actor_user_id` nullable
- `action`
- `target_type`
- `target_id`
- `metadata_json`
- `created_at`

---

6. Core CRM Entities

These are shared CRM primitives that most modules can build upon.

6.1 contacts

Represents an individual person.

Suggested Fields

- `id`
- `tenant_id`
- `first_name`
- `last_name`
- `full_name`
- `email`
- `phone`
- `title`
- `company_id` nullable
- `owner_user_id`
- `status`
- `source`
- `metadata_json`
- `created_by`
- `updated_by`
- `created_at`
- `updated_at`
- `deleted_at`

6.2 companies

Represents an organization or account.

Suggested Fields

- `id`
- `tenant_id`
- `name`
- `domain`
- `phone`
- `industry`
- `owner_user_id`
- `status`
- `metadata_json`
- `created_by`
- `updated_by`
- `created_at`
- `updated_at`
- `deleted_at`

6.3 leads

Represents pre-qualified prospects.

Suggested Fields

- `id`
- `tenant_id`
- `contact_id` nullable
- `company_id` nullable
- `owner_user_id`
- `source`
- `status`
- `score`
- `metadata_json`
- `created_by`

- `updated_by`
- `created_at`
- `updated_at`
- `deleted_at`

6.4 opportunities

Represents pipeline deals or active sales opportunities.

Suggested Fields

- `id`
- `tenant_id`
- `name`
- `contact_id` nullable
- `company_id` nullable
- `pipeline_id`
- `pipeline_stage_id`
- `owner_user_id`
- `amount`
- `close_date`
- `status`
- `probability`
- `metadata_json`
- `created_by`
- `updated_by`
- `created_at`
- `updated_at`
- `deleted_at`

6.5 pipelines

Represents a process flow.

Suggested Fields

- `id`

- `tenant_id`
- `name`
- `entity_type`
- `is_default`
- `created_at`
- `updated_at`

Examples:

- sales pipeline
- intake pipeline
- recruiting pipeline

6.6 pipeline_stages

Represents stages within a pipeline.

Suggested Fields

- `id`
- `tenant_id`
- `pipeline_id`
- `name`
- `slug`
- `position`
- `probability`
- `is_closed`
- `created_at`
- `updated_at`

6.7 tasks

Represents work items.

Suggested Fields

- `id`
- `tenant_id`

- `title`
- `description`
- `assigned_user_id`
- `owner_user_id`
- `related_type`
- `related_id`
- `due_at`
- `status`
- `priority`
- `completed_at`
- `created_by`
- `updated_by`
- `created_at`
- `updated_at`
- `deleted_at`

Use `related_type` and `related_id` for polymorphic relationships.

6.8 activities

Represents interactions or timeline events.

Suggested Fields

- `id`
- `tenant_id`
- `type`
- `subject`
- `description`
- `related_type`
- `related_id`
- `actor_user_id`
- `occurred_at`
- `metadata_json`
- `created_at`
- `updated_at`

Examples:

- call logged
- email sent
- meeting held
- status changed

6.9 notes

Represents freeform notes.

Suggested Fields

- `id`
- `tenant_id`
- `body`
- `related_type`
- `related_id`
- `author_user_id`
- `visibility`
- `created_at`
- `updated_at`
- `deleted_at`

6.10 files

Represents uploaded files.

Suggested Fields

- `id`
- `tenant_id`
- `storage_provider`
- `storage_key`
- `file_name`
- `mime_type`
- `file_size`
- `related_type`
- `related_id`

- `uploaded_by`
- `created_at`
- `updated_at`

6.11 tags

Represents tenant-defined labels.

Suggested Fields

- `id`
- `tenant_id`
- `name`
- `color`
- `entity_type`
- `created_at`
- `updated_at`

6.12 taggables

Maps tags to entities.

Suggested Fields

- `id`
- `tenant_id`
- `tag_id`
- `related_type`
- `related_id`

---

7. Module-Owned Entities

Modules can introduce domain-specific business records.

These tables should still follow tenant rules.

7.1 Example: Legal Module

legal_cases

- `id`
- `tenant_id`
- `contact_id`
- `company_id` nullable
- `assigned_user_id`
- `case_number`
- `case_type`
- `status`
- `opened_at`
- `closed_at`
- `metadata_json`
- `created_at`
- `updated_at`

legal_documents

- `id`
- `tenant_id`
- `case_id`
- `file_id`
- `document_type`
- `status`
- `created_at`
- `updated_at`

7.2 Example: Real Estate Module

re_properties

- `id`
- `tenant_id`
- `address_line_1`
- `address_line_2`
- `city`

- `state`
- `postal_code`
- `listing_status`
- `price`
- `assigned_user_id`
- `metadata_json`
- `created_at`
- `updated_at`

re_showings

- `id`
- `tenant_id`
- `property_id`
- `contact_id`
- `scheduled_at`
- `status`
- `notes`
- `created_at`
- `updated_at`

7.3 Example: Event Module

event_projects

- `id`
- `tenant_id`
- `name`
- `client_contact_id`
- `client_company_id`
- `status`
- `budget`
- `event_date`
- `metadata_json`
- `created_at`
- `updated_at`

event_vendors

- `id`
- `tenant_id`
- `project_id`
- `name`
- `category`
- `contact_info_json`
- `status`
- `created_at`
- `updated_at`

---

8. Custom Fields Strategy

A modular CRM needs structured flexibility without exploding the schema.

Recommended Strategy

Support custom fields on selected entities through metadata-driven definitions and typed value storage.

8.1 custom_field_definitions

Defines a field available for an entity within a tenant.

Suggested Fields

- `id`
- `tenant_id`
- `entity_type`
- `module_slug` nullable
- `key`
- `label`
- `field_type`
- `is_required`
- `is_searchable`
- `is_filterable`

- `default_value_json`
- `validation_rules_json`
- `settings_json`
- `position`
- `created_at`
- `updated_at`

Examples of `field_type`:

- text
- textarea
- number
- currency
- boolean
- date
- datetime
- select
- multiselect
- url

8.2 custom_field_values

Stores per-record values.

Suggested Fields

- `id`
- `tenant_id`
- `custom_field_definition_id`
- `entity_type`
- `entity_id`
- `value_text`
- `value_number`
- `value_boolean`
- `value_date`
- `value_datetime`
- `value_json`
- `created_at`

- `updated_at`

This typed approach is more query-friendly than a single raw JSON blob.

8.3 Recommended Rules for Custom Fields

Custom fields should support:

- tenant-level configuration
- per-entity definitions
- validation rules
- UI rendering metadata
- search/filter support for selected field types

Custom fields should not replace module-owned relational tables for high-value workflows.

---

9. Settings and Configuration Tables

Configuration should be normalized where helpful, and JSON-backed where flexibility matters more than querying.

9.1 tenant_settings

Stores tenant-level configuration.

Suggested Fields

- `id`
- `tenant_id`
- `key`
- `value_json`
- `created_at`
- `updated_at`

Examples:

- default locale

- time format
- notification preferences

9.2 branding_settings

Stores tenant branding.

Suggested Fields

- `id`
- `tenant_id`
- `logo_file_id` nullable
- `favicon_file_id` nullable
- `primary_color`
- `secondary_color`
- `theme_json`
- `created_at`
- `updated_at`

9.3 module_settings

Stores configuration for installed modules.

Suggested Fields

- `id`
- `tenant_module_id`
- `key`
- `value_json`
- `created_at`
- `updated_at`

9.4 saved_views

Stores list/table/filter presets.

Suggested Fields

- `id`
- `tenant_id`
- `user_id` nullable
- `entity_type`
- `name`
- `filters_json`
- `sort_json`
- `columns_json`
- `visibility`
- `created_at`
- `updated_at`

---

10. Relationship Strategy

Relationships should be explicit and consistent.

10.1 Prefer Foreign Keys for Stable Relationships

Use direct foreign keys when the related entity is known and stable.

Examples:

- `contacts.company_id -> companies.id`
- `opportunities.pipeline_id -> pipelines.id`
- `legal_documents.case_id -> legal_cases.id`

10.2 Use Polymorphic Relationships Carefully

Polymorphic patterns are useful for cross-cutting entities like:

- tasks
- notes
- activities
- files

- tags

Examples:

- `related_type`
- `related_id`

Use them where flexibility is worth the tradeoff.

Do not overuse polymorphic relationships for everything.

10.3 Cross-Module Relationships

Modules should be allowed to reference:

- core CRM entities
- their own entities
- approved public entities from other modules

This should be governed through entity registration and clear module contracts.

---

11. Indexing Strategy

The shared multi-tenant model requires strong indexing.

Every tenant-owned table should generally have indexes on:

- `tenant_id`
- common filter fields
- foreign keys
- status fields
- owner/assignee fields
- date fields used in list views

Examples:

contacts

- `(tenant_id, email)`
- `(tenant_id, full_name)`

- `(tenant_id, owner_user_id)`

opportunities

- `(tenant_id, pipeline_id, pipeline_stage_id)`
- `(tenant_id, owner_user_id)`
- `(tenant_id, close_date)`

tasks

- `(tenant_id, assigned_user_id, status)`
- `(tenant_id, due_at)`

module tables

- indexes should always include `tenant_id` as a leading dimension where appropriate

---

12. Soft Delete Strategy

Soft deletes are recommended for major user-facing records.

Use soft deletes for:

- contacts
- companies
- leads
- opportunities
- tasks
- notes
- module business entities where recovery matters

Avoid soft deletes for purely join or mapping tables unless necessary.

Examples to avoid soft deletes on:

- role_permissions
- taggables

---

13. Auditability and Change Tracking

Important changes should be trackable.

Recommended layers:

Basic Row Metadata

- `created_at`
- `updated_at`
- `created_by`
- `updated_by`

Audit Logs

Track key actions such as:

- record created
- record deleted
- module installed
- permission changed
- settings updated

Optional Change History

For critical entities, store field-level change history.

Examples:

- opportunity stage changes
- case status changes
- ownership changes

---

14. Search Strategy

The data model should support both:

- direct relational queries
- unified search experiences

Searchable entities should register:

- entity type
- display label

- searchable fields
- preview fields
- permission requirements

The platform can later build:

- database-backed search
- full-text search
- search indexing pipelines
- module-aware search providers

---

15. Module Schema Governance

Modules must follow governance rules to avoid schema chaos.

15.1 Naming Rules

Module tables should use prefixed or namespaced table names.

Examples:

- `legal_cases`
- `re_properties`
- `event_projects`

15.2 Migration Rules

Every module migration must define:

- version
- required platform version
- dependencies
- upgrade path
- rollback expectations if supported

Platform-controlled module installation should run migrations safely.

15.3 Public vs Private Entities

Modules should declare which entities are:

- internal/private
- publicly referenceable by other modules
- searchable
- reportable

This prevents accidental tight coupling across modules.

---

16. Suggested V1 Table Groups

Below is a practical V1 grouping.

16.1 Platform Tables

- users
- tenants
- memberships
- roles
- permissions
- role_permissions
- domains
- subscriptions
- modules
- module_versions
- tenant_modules
- audit_logs

16.2 Core CRM Tables

- contacts
- companies
- leads

- opportunities
- pipelines
- pipeline_stages
- tasks
- activities
- notes
- files
- tags
- taggables

16.3 Configuration Tables

- tenant_settings
- branding_settings
- module_settings
- saved_views
- custom_field_definitions
- custom_field_values

16.4 Module Tables

Varies by installed module, for example:

- legal_cases
- legal_documents
- re_properties
- re_showings
- event_projects
- event_vendors

---

17. Recommended V1 Data Model Decisions

To keep V1 practical and scalable:

Tenant Strategy

Shared relational database with `tenant_id` on all tenant-owned tables.

CRM Core

Define reusable core entities: contacts, companies, leads, opportunities, tasks, activities, notes, files.

Module Data

Allow module-owned namespaced tables.

Flexibility

Use custom field definitions and typed custom field values.

JSON Usage

Use JSON for config and metadata, not for core transactional entities.

Relationships

Use direct foreign keys for stable relationships; polymorphic only where it clearly adds value.

Auditing

Include `created_by`, `updated_by`, timestamps, and audit logs for key actions.

Governance

Require module schema conventions and migration control.

---

18. Example Conceptual ERD Groups

A simplified conceptual structure looks like this:

Platform

- users
- tenants
- memberships
- subscriptions
- modules
- tenant_modules

Core CRM

- contacts
- companies
- opportunities
- tasks
- activities
- notes

Config

- tenant_settings
- branding_settings
- custom_field_definitions
- custom_field_values

Module Layer

- legal_cases
- re_properties
- event_projects
- other module-owned entities

Core CRM entities connect upward to the tenant and outward to module entities.

---

19. Final Summary

The data model should give the platform a strong relational backbone while still supporting modular growth.

The best long-term shape is:

- a stable platform schema
- a reusable CRM core
- namespaced module-owned tables
- tenant-aware relationships everywhere
- controlled flexibility through custom fields and config tables

This balance will make the platform easier to:

- query
- secure
- extend
- report on
- evolve across multiple industry verticals

Without this balance, the system risks becoming either too rigid to customize or too unstructured to scale.

20. Conceptual Entity Relationship Overview

The following conceptual relationships illustrate how the core system connects.

This structure ensures:

- **Platform identity and membership are separated**
- **CRM primitives remain reusable**
- **Modules extend the system safely**
- **Tenant context anchors everything**

---

21. Record Lifecycle Model

Most CRM entities follow a common lifecycle pattern.

Example lifecycle for a **contact record**.

1. Creation

A record is created with:

- tenant_id
- created_by
- timestamps
- owner_user_id

Example:

```
Contact Created
→ emit event: contact.created
→ automation may trigger
→ activity logged
```

2. Updates

Records may update fields such as:

- ownership
- status
- relationships
- custom fields

Each update should optionally produce:

```
contact.updated
```

Audit metadata may include:

- actor
- changed fields
- timestamp

---

3. Related Records

Other entities may attach to the contact:

- tasks
- notes
- activities
- opportunities
- module entities

Example:

```
Contact
 ├── Tasks
 ├── Notes
 ├── Activities
 ├── Opportunities
 └── Module Entities
```

---

4. Soft Delete

Most CRM entities should support recovery.

Example:

```
deleted_at != null
```

This allows:

- undo actions
- audit traceability
- safer module interactions

---

22. Module Data Contracts

Modules must follow specific rules when introducing entities.

22.1 Entity Registration

Each module should register its entities with metadata.

Example:

```
{
  "entity": "legal_cases",
  "displayName": "Legal Case",
  "ownerField": "assigned_user_id",
  "searchable": true,
  "permissions": [
    "legal_cases.read",
    "legal_cases.write"
  ]
}
```